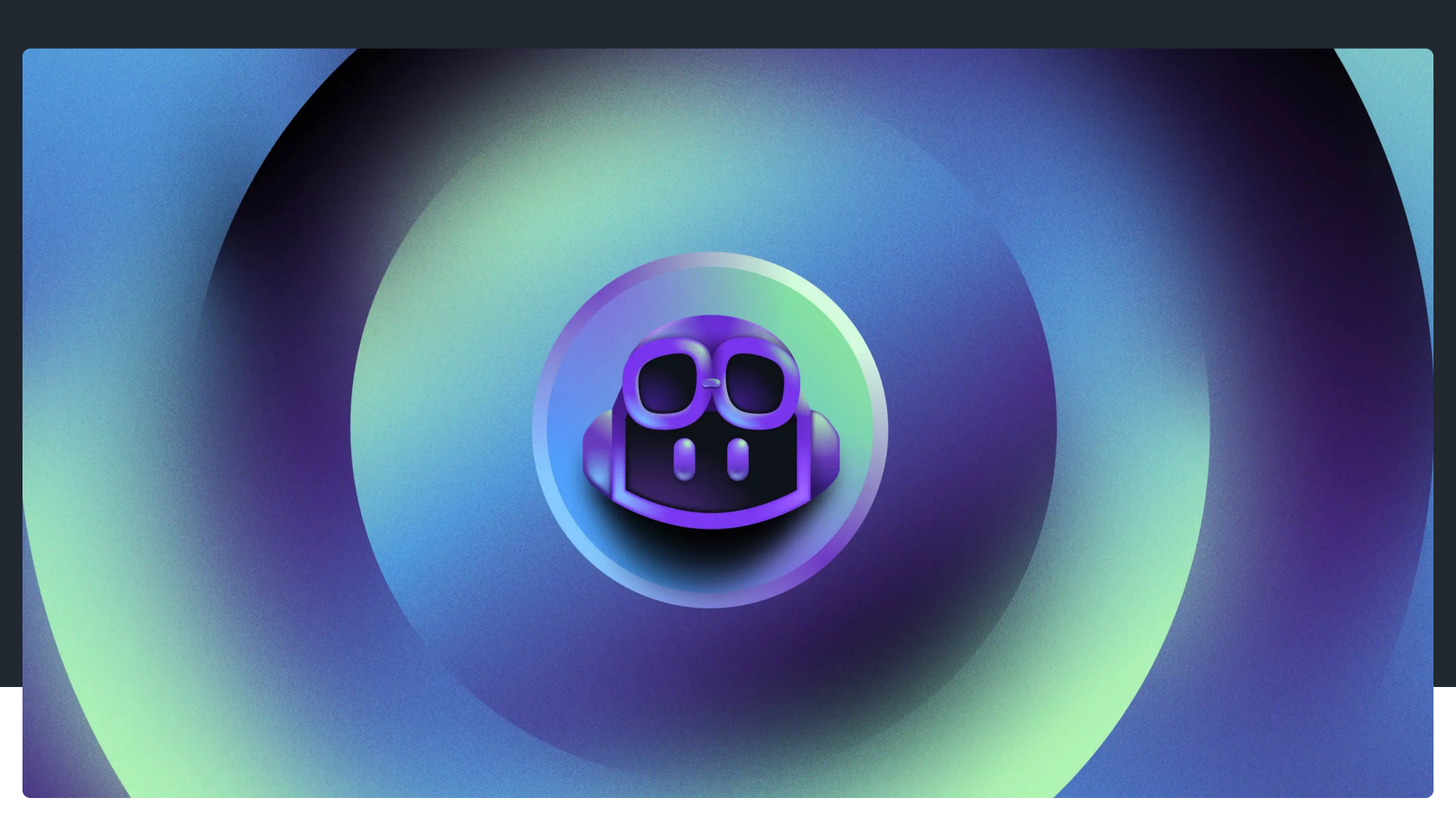
How to write better prompts for GitHub Copilot

In this prompt guide for GitHub Copilot, two GitHub developer advocates, Rizel and Michelle, will share examples and best practices for communicating your desired results to the AI pair programmer.



Rizel Scarlett & Michelle Duke
June 20, 2023 | Updated February 26, 2025 | 25 minutes

AI Insights
GitHub Copilot
prompt writing
tutorial

More on **AI Insights**

Survey: The AI wave continues to grow on software development teams

We surveyed 2,000 people on software development teams at enterprises in the U.S., Brazil, India, and Germany about the use, experience, and expectations around generative AI tools in software development.

Kyle Daigle & GitHub Staff

Unlocking the power of unstructured data with RAG

Unstructured data holds valuable information about codebases, organizational best practices, and customer feedback. Here are some ways you can leverage it with RAG, or retrieval-augmented generation.

Nicole Choi

Generative AI coding tools are transforming the way developers approach daily coding tasks. From documenting our codebases to generating unit tests, these tools are helping to accelerate our workflows. However, just like with any emerging tech, there's always a learning curve. As a result, developers—beginners and experienced alike—sometimes feel frustrated when AI-powered coding assistants don't generate the output they want. (Feel familiar?)

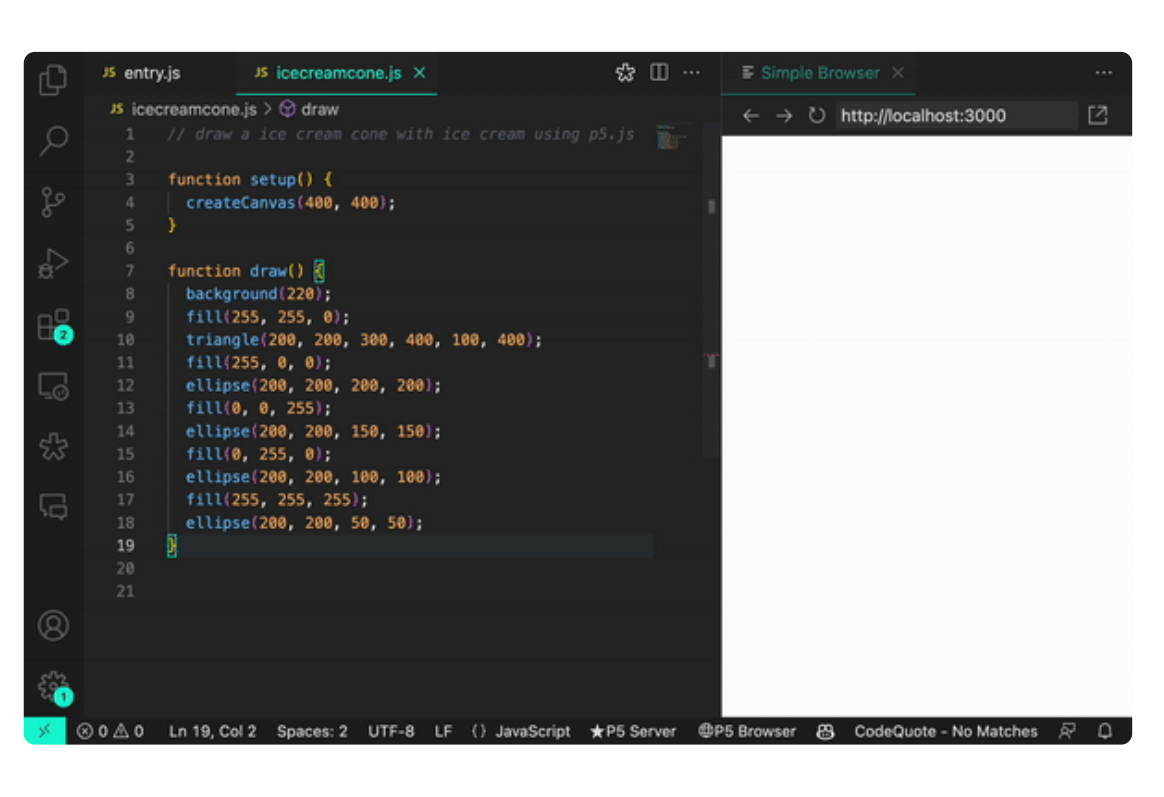
For example, when asking GitHub Copilot to draw an ice cream cone 🍦 using p5.js, a JavaScript library for creative coding, we kept receiving irrelevant suggestions—or sometimes no suggestions at all. But when we learned more about the way that GitHub Copilot processes information, we realized that we had to adjust the way we communicated with it.

Get the latest on GitHub Copilot

GitHub Copilot is always adding new features and functionality. Check out these resources to keep up to date:

- Our comprehensive guide on how to use GitHub Copilot with real-world examples
- The GitHub Copilot tag and GitHub Copilot category on the GitHub Blog
- The GitHub Copilot documentation and the GitHub Copilot Chat cookbook

Here's an example of GitHub Copilot generating an irrelevant solution:



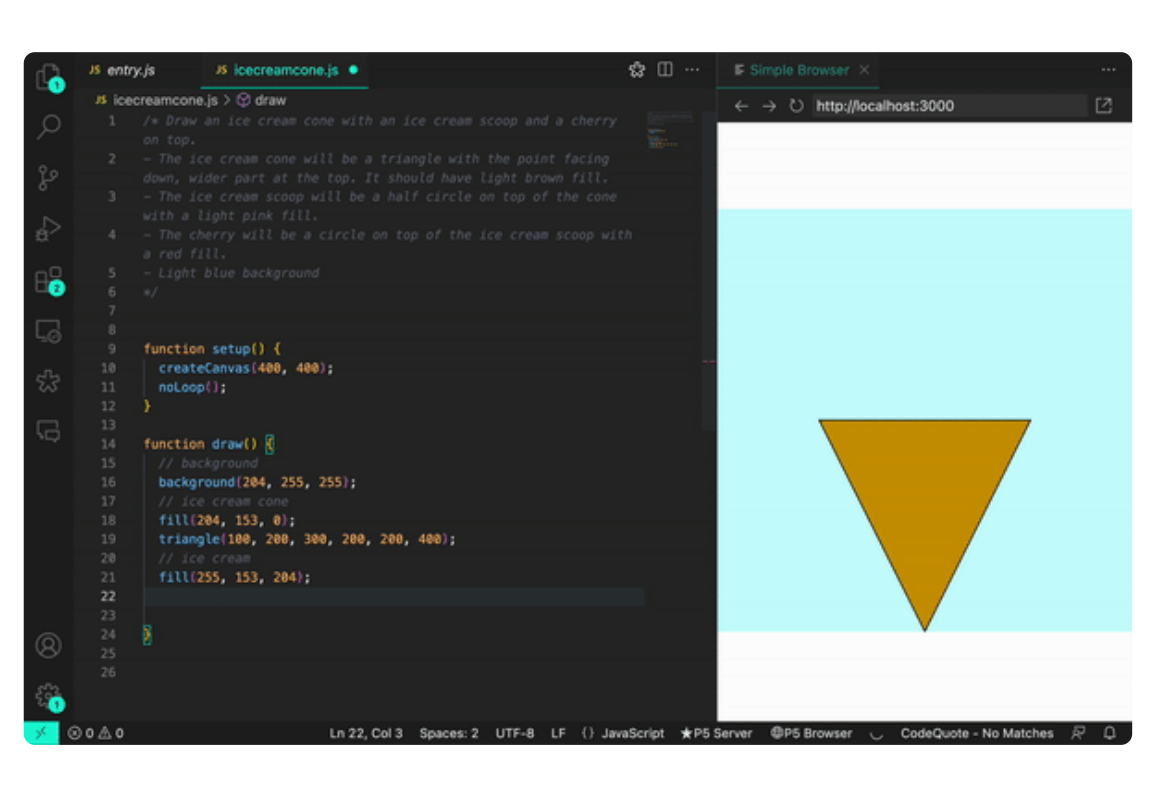
```
// draw a ice cream cone with ice cream using p5.js

function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
  fill(255, 255, 0);
  triangle(200, 200, 300, 400, 100, 400);
  fill(255, 0, 0);
  ellipse(200, 200, 200, 200);
  fill(0, 0, 255);
  ellipse(200, 200, 150, 150);
  fill(0, 255, 0);
  ellipse(200, 200, 100, 100);
  fill(255, 255, 255);
  ellipse(200, 200, 50, 50);
}
```

entry.js
icecreamcone.js
icecreamcone.js > draw
Simple Browser
http://localhost:3000
Ln 19, Col 2 Spaces: 2 UTF-8 LF {} JavaScript ★P5 Server P5 Browser CodeQuote - No Matches

When we adjusted our prompt, we were able to generate more accurate results:



```
/* Draw an ice cream cone with an ice cream scoop and a cherry on top.
- The ice cream cone will be a triangle with the point facing down, wider part at the top. It should have light brown fill.
- The ice cream scoop will be a half circle on top of the cone with a light pink fill.
- The cherry will be a circle on top of the ice cream scoop with a red fill.
- Light blue background
*/


function setup() {
  createCanvas(400, 400);
  noLoop();
}

function draw() {
  // background
  background(204, 255, 255);
  // ice cream cone
  fill(204, 153, 0);
  triangle(100, 200, 300, 200, 200, 400);
  // ice cream
  fill(255, 153, 204);


}
```

entry.js
icecreamcone.js
icecreamcone.js > draw
Simple Browser
http://localhost:3000
Ln 22, Col 3 Spaces: 2 UTF-8 LF {} JavaScript ★P5 Server P5 Browser CodeQuote - No Matches

We're both developers and AI enthusiasts ourselves. I, Rizel, have used GitHub Copilot to build a browser extension, rock, paper, scissors game, and to send a Tweet. And I, Michelle, launched an AI company in 2016. We're both developer advocates at GitHub and love to share our top tips for working with GitHub Copilot.

In this guide for GitHub Copilot, we'll cover:

- What exactly a prompt is and what prompt engineering is, too (hint: it depends on whether you're talking to a developer or a machine learning researcher).
- Three best practices and three additional tips for prompt crafting with GitHub Copilot.
- An example where you can try your hand at prompting GitHub Copilot to assist you in building a browser extension.

Progress over perfection

Even with our experience using AI, we recognize that everyone is in a trial and error phase with generative AI technology. We also know the challenge of providing generalized prompt-crafting tips because models vary, as do the individual problems that developers are working on. This isn't an end-all, be-all guide. Instead, we're sharing what we've learned about prompt crafting to accelerate collective learning during this new age of software development.

What's a prompt and what is prompt engineering?

It depends on who you talk to.

In the context of generative AI coding tools, **a prompt can mean different things, depending on whether you're asking machine learning (ML) researchers** who are building and fine-tuning these tools, **or developers** who are using them in their IDEs.

For this guide, we'll define the terms from the point of view of a developer who's using a generative AI coding tool in the IDE. But to give you the full picture, we also added the ML researcher definitions below in our chart.

	Prompts	Prompt engineering	Context
Developer	**Code blocks, individual lines of code, or natural language comments a developer writes** to generate a specific suggestion from GitHub Copilot	**Providing instructions or comments in the IDE** to generate specific coding suggestions	**Details that are provided by a developer** to specify the desired output from a generative AI coding tool
ML researcher	**Compilation of IDE code and relevant context** (IDE comments, code in open files, etc.) **that is continuously generated by algorithms** and sent to the model of a generative AI coding tool	Creating algorithms that will generate prompts (compilations of IDE code and context) for a large language model	**Details** (like data from your open files and code you've written before and after the cursor) **that algorithms send** to a large language model (LLM) as additional information about the code

3 best practices for prompt crafting with GitHub Copilot

1. Set the stage with a high-level goal. 🖼️

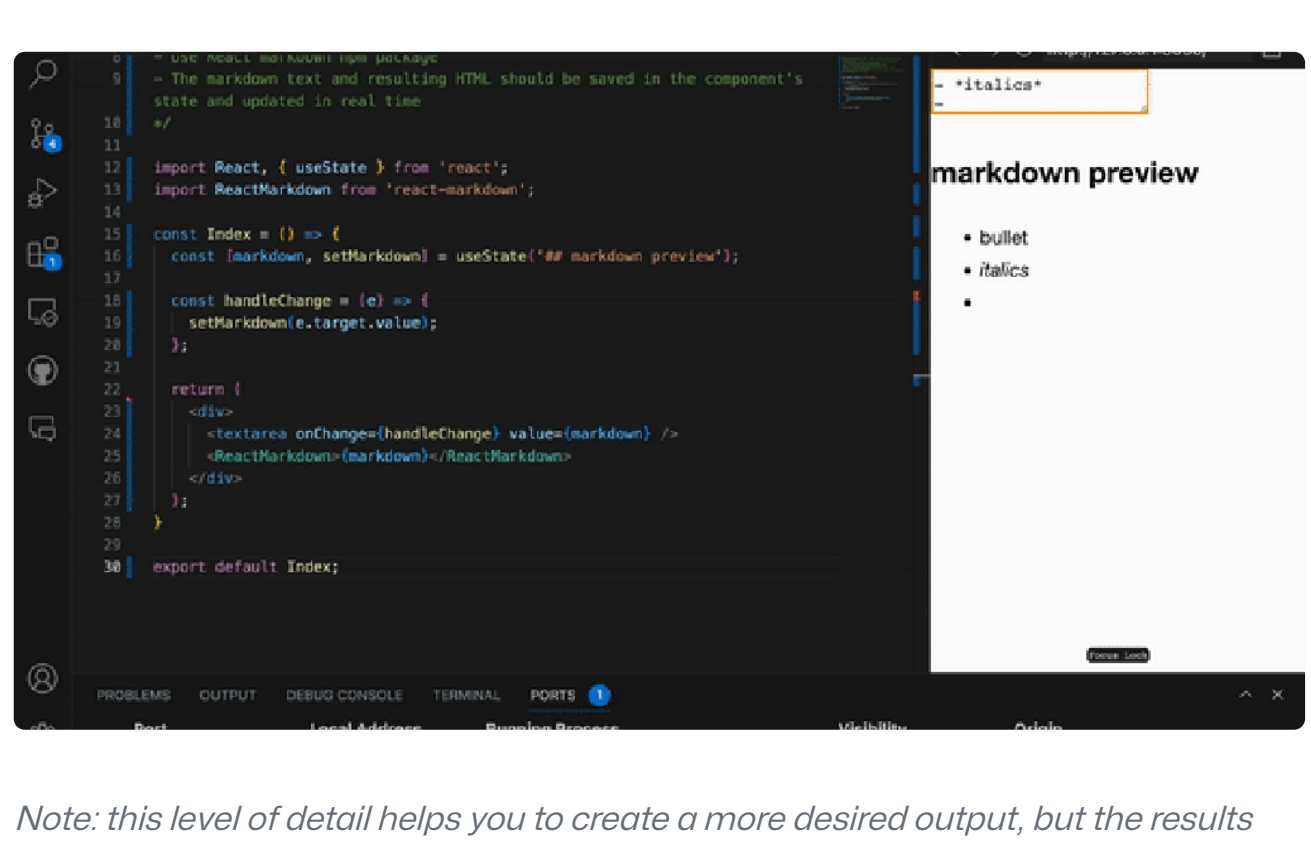
This is most helpful if you have a blank file or empty codebase. In other words, if GitHub Copilot has zero context of what you want to build or accomplish, setting the stage for the AI pair programmer can be really useful. It helps to prime GitHub Copilot with a big picture description of what you want it to generate—before you jump in with the details.

Think of the process as having a conversation with someone: *How should I break down the problem so we can solve it together? How would I approach pair programming with this person?*

For example, when building a markdown editor in Next.js, we could write a comment like this

```
/*
Create a basic markdown editor in Next.js with the following features:
- Use react hooks
- Create state for markdown with default text "type markdown here"
- A text area where users can write markdown
- Show a live preview of the markdown text as I type
- Support for basic markdown syntax like headers, bold, italics
- Use React markdown npm package
- The markdown text and resulting HTML should be saved in the component's sta
*/
```

This will prompt GitHub Copilot to generate the following code and produce a very simple, unstyled but functional markdown editor in less than 30 seconds. We can use the remaining time to style the component:



```
- The markdown text and resulting HTML should be saved in the component's
state and updated in real time
*/

import React, { useState } from 'react';
import ReactMarkdown from 'react-markdown';

const Index = () => {
  const [markdown, setMarkdown] = useState('## markdown preview');

  const handleChange = (e) => {
    setMarkdown(e.target.value);
  };

  return (
    <div>
      <textarea onChange={handleChange} value={markdown} />
      <ReactMarkdown>{markdown}</ReactMarkdown>
    </div>
  );
}

export default Index;
```

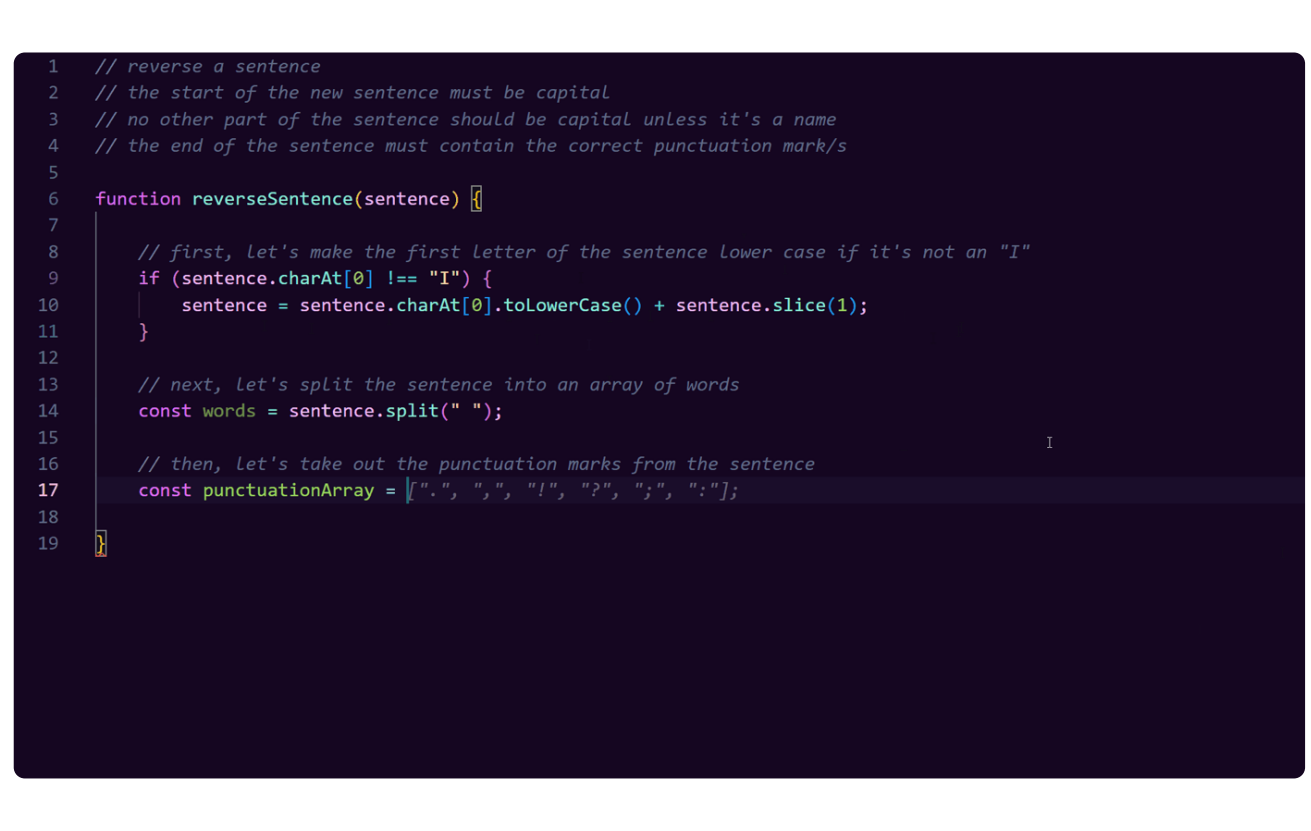
Note: this level of detail helps you to create a more desired output, but the results may still be non-deterministic. For example, in the comment, we prompted GitHub Copilot to create default text that says "type markdown here" but instead it generated "markdown preview" as the default words.

2. Make your ask simple and specific. Aim to receive a short output from GitHub Copilot. 🗨️

Once you communicate your main goal to the AI pair programmer, **articulate the logic and steps it needs to follow for achieving that goal**. GitHub Copilot better understands your goal when you break things down. (Imagine you're writing a recipe. You'd break the cooking process down into discrete steps–not write a paragraph describing the dish you want to make.)

Let GitHub Copilot generate the code after each step, rather than asking it to generate a bunch of code all at once.

Here's an example of us providing GitHub Copilot with step-by-step instructions for reversing a function:



```
// reverse a sentence
// the start of the new sentence must be capital
// no other part of the sentence should be capital unless it's a name
// the end of the sentence must contain the correct punctuation mark/s

function reverseSentence(sentence) {

    // first, let's make the first letter of the sentence lower case if it's not an "I"
    if (sentence.charAt[0] !== "I") {
        sentence = sentence.charAt[0].toLowerCase() + sentence.slice(1);
    }

    // next, let's split the sentence into an array of words
    const words = sentence.split(" ");

    // then, let's take out the punctuation marks from the sentence
    const punctuationArray = [".", ",", "!", "?", ";", ":"];

}
```

3. Give GitHub Copilot an example or two. ✍️

Learning from examples is not only useful for humans, but also for your AI pair programmer. For instance, we wanted to extract the names from the array of data below and store it in a new array:

```
const data = [
[
{ name: 'John', age: 25 },
{ name: 'Jane', age: 30 }
],
[
{ name: 'Bob', age: 40 }
]
];
```

When we didn't show GitHub Copilot an example …

```
// Map through an array of arrays of objects to transform data
const data = [
[
{ name: 'John', age: 25 },
{ name: 'Jane', age: 30 }
],
[
{ name: 'Bob', age: 40 }
]
];

const mappedData = data.map(x => [x.name](http://x.name/));

console.log(mappedData);

// Results: [undefined, undefined]
```

It generated an incorrect usage of map:

```
const mappedData = data.map(x => [x.name](http://x.name/));

console.log(mappedData);

// Results: [undefined, undefined]
```

By contrast, when we did provide an example …

```
// Map through an array of arrays of objects
// Example: Extract names from the data array
// Desired outcome: ['John', 'Jane', 'Bob']
const data = [
[{ name: 'John', age: 25 }, { name: 'Jane', age: 30 }],
[{ name: 'Bob', age: 40 }]
];

const mappedData = data.flatMap(sublist => sublist.map(person => person.name)

console.log(mappedData);
```

We received our desired outcome.

```
const mappedData = data.flatMap(sublist => sublist.map(person => person.name)

console.log(mappedData);
// Results: ['John', 'Jane', 'Bob']
```

Read more about common approaches to AI training, such as zero-shot, one-shot, and few-shot learning.

Three additional tips for prompt crafting with GitHub Copilot

Here are three additional tips to help guide your conversation with GitHub Copilot.

1. Experiment with your prompts.

Just how conversation is more of an art than a science, so is prompt crafting. So, if you don't receive what you want on the first try, recraft your prompt by following the best practices above.

For example, the prompt below is vague. It doesn't provide any context or boundaries for GitHub Copilot to generate relevant suggestions.

```
# Write some code for grades.py
```

We iterated on the prompt to be more specific, but we still didn't get the exact result we were looking for. This is a good reminder that adding specificity to your prompt is harder than it sounds. It's difficult to know, from the start, which details you should include about your goal to generate the most useful suggestions from GitHub Copilot. That's why we encourage experimentation.

The version of the prompt below is more specific than the one above, but it doesn't clearly define the input and output requirements.

```
# Implement a function in grades.py to calculate the average grade
```

We experimented with the prompt once more by setting boundaries and outlining what we wanted the function to do. We also rephrased the comment so the function was more clear (giving GitHub Copilot a clear intention to verify against).

This time, we got the results we were looking for.

```
t of grades as input and returns the average grade as a floating-point number
```

2. Keep a couple of relevant tabs open.

We don't have an exact number of tabs that you should keep open to help GitHub Copilot contextualize your code, but from our experience, we've found that one or two is helpful.

GitHub Copilot uses a technique called neighboring tabs that allows the AI pair programmer to contextualize your code by processing all of the files open in your IDE instead of just the single file you're working on. However, it's not guaranteed that GitHub Copilot will deem all open files as necessary context for your code.

3. Use good coding practices.

That includes providing descriptive variable names and functions, and following consistent coding styles and patterns. We've found that working with GitHub Copilot encourages us to follow good coding practices we've learned throughout our careers.

For example, here we used a descriptive function name and followed the codebase's patterns of leveraging snake case.

```
def authenticate_user(username, password):
```

As a result, GitHub Copilot generated a relevant code suggestion:

```
def authenticate_user(username, password):
# Code for authenticating the user
if is_valid_user(username, password):
generate_session_token(username)
return True
else:
return False
```

Compare this to the example below, where we introduced an inconsistent coding style and poorly named our function.

```
def rndpwd(l):
```

Instead of suggesting code, GitHub Copilot generated a comment that said, "Code goes here."

```
def rndpwd(l):
# Code goes here
```

Stay smart

The LLMs behind generative AI coding tools are designed to find and extrapolate patterns from their training data, apply those patterns to existing language, and then produce code that follows those patterns. Given the sheer scale of these models, they might generate a code sequence that doesn't even exist yet. **Just as you would review a colleague's code, you should always assess, analyze, and validate AI-generated code.**

A practice example 🧑‍💻

Try your hand at prompting GitHub Copilot to build a browser extension.

To get started, you'll need to have GitHub Copilot installed and open in your IDE.

More generative AI prompt crafting guides

- A beginner's guide to prompt engineering with GitHub Copilot
- Prompt engineering for AI
- How GitHub Copilot is getting better at understanding your code

Tags: AI Insights GitHub Copilot prompt writing tutorial

Written by

Rizel Scarlett
@blackgirlbytes

Michelle Duke
@MishManners

I'm a Content Producer working in tech & innovation. Known as the "Hackathon Queen" 👑 I'm on the GitHub DevRel team and love sharing stories from our amazing community of developers.

Related posts

Video: How to run dependency audits with GitHub Copilot

Learn to automate dependency management using GitHub Copilot, GitHub Actions, and Dependabot to eliminate manual checks, improve security, and save time for what really matters.

Andrea Griffiths

Supporting the next generation of developers

Here's your opportunity to empower the teen in your life to get a start in open source development.

Morgan Ersery

How we built the GitHub Skyline CLI extension using GitHub

GitHub uses GitHub to build GitHub, and our CLI extensions are no exception. Read on to find out how we built the GitHub Skyline CLI extension using GitHub!

Chris Reddington

Explore more from GitHub

Docs

Everything you need to master GitHub, all in one place.

Go to Docs

The ReadME Project

Stories and voices from the developer community.

Learn more

GitHub Copilot

Don't fly solo. Try 30 days for free.

Learn more →

Work at GitHub!

Check out our current job openings.

Apply now

© 2025 GitHub, Inc. Terms Privacy Manage Cookies Do not share my personal information